

APRESENTAÇÃO DE TÉCNICAS ALGORÍTMICAS E COMPUTABILIDADE

TEMA DO PROJETO: BENCHMARK DE ALGORITMOS DE
ORDENAÇÃO

Trabalho realizado por:

Afonso Romão, A114623

Armand Neto A103267

Gonçalo Teixeira A

ESCOPO DO PROJETO:

- Este projeto visa estudar o comportamento assintótico de diferentes tipos de algoritmos de ordenação.
- Na algoritmia do programa final, recorreram-se a arrays de tamanhos entre 100 a 10000 elementos de um ciclo for no seu benchmark.

INSERTION SORT -

- CÓDIGO:

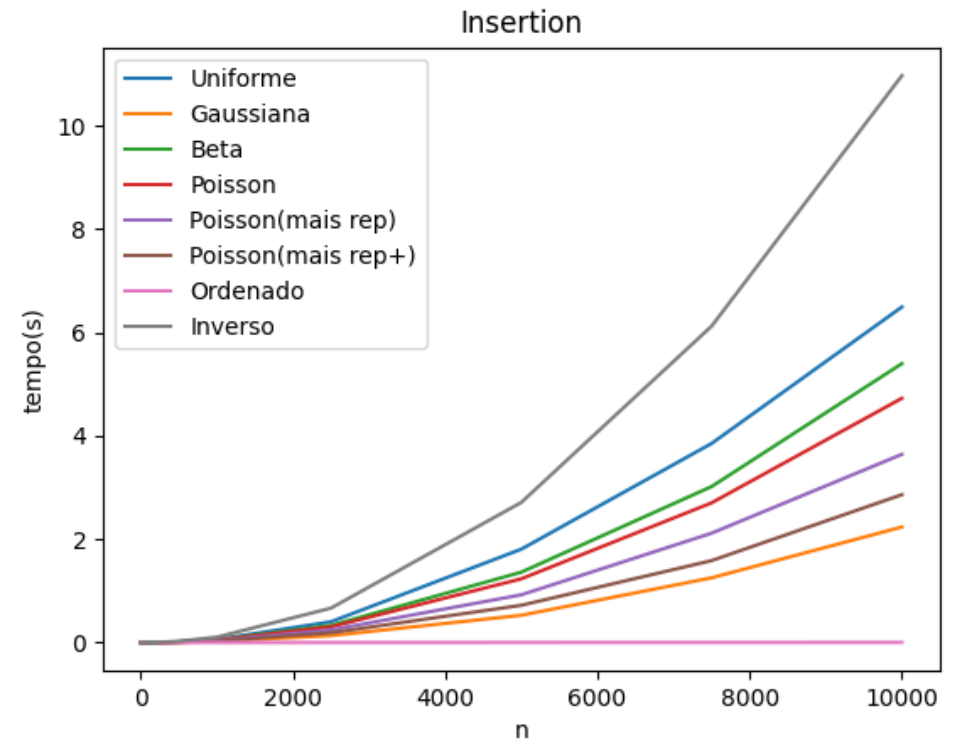
```
def insertionSort(V):  
    N = len(V)  
    for j in range(1, N):  
        key = V[j]  
        i = j - 1  
        while i >= 0 and V[i] >  
key:  
            V[i + 1] = V[i]  
            i -= 1  
        V[i + 1] = key
```

MELHOR CASO:
ORDENADO - $O(N)$

PIOR CASO: INVERSO
 $O(N^2)$

CASO MÉDIO: $O(N^2)$

GRÁFICO:



SHELL SORT -

MELHOR CASO:
ORDENADO - $O(N \log N)$

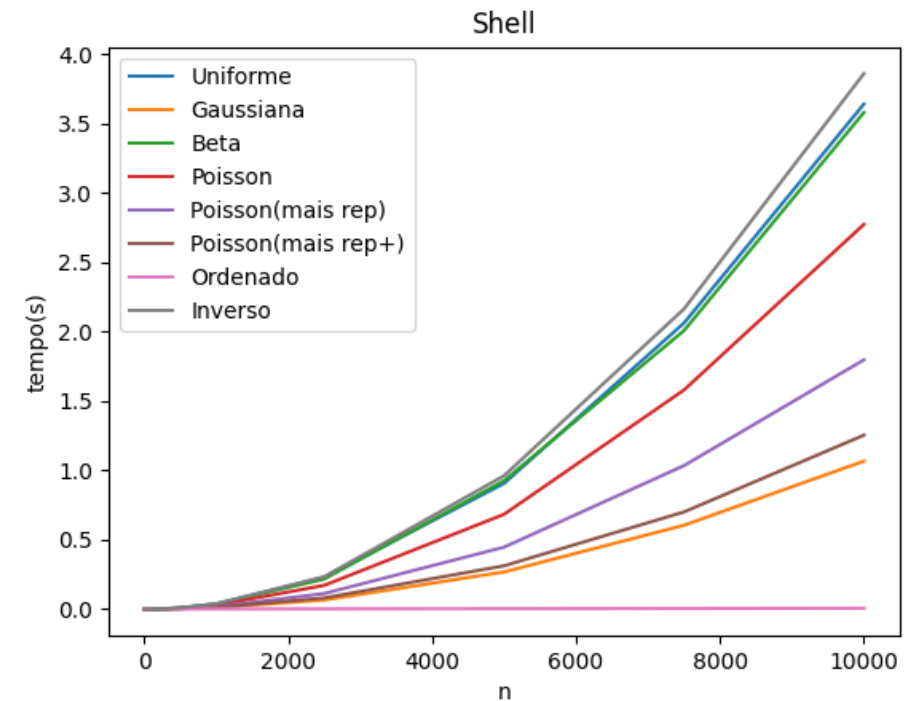
PIOR CASO: INVERSO - $O(N^2)$

CASO MÉDIO: $O(N^{1.25}) - O(N^{1.5})$

• CÓDIGO:

```
def shellSort(V, gaps):  
    N = len(V)  
    for h in gaps:  
        for i in range(h, N):  
            key = V[i]  
            j = i  
            while j >= h and V[j - h] > key:  
                V[j] = V[j - h]  
                j -= h  
            V[j] = key
```

GRÁFICO:



BUBBLE SORT -

MELHOR CASO: ORDENADO - $O(N)$

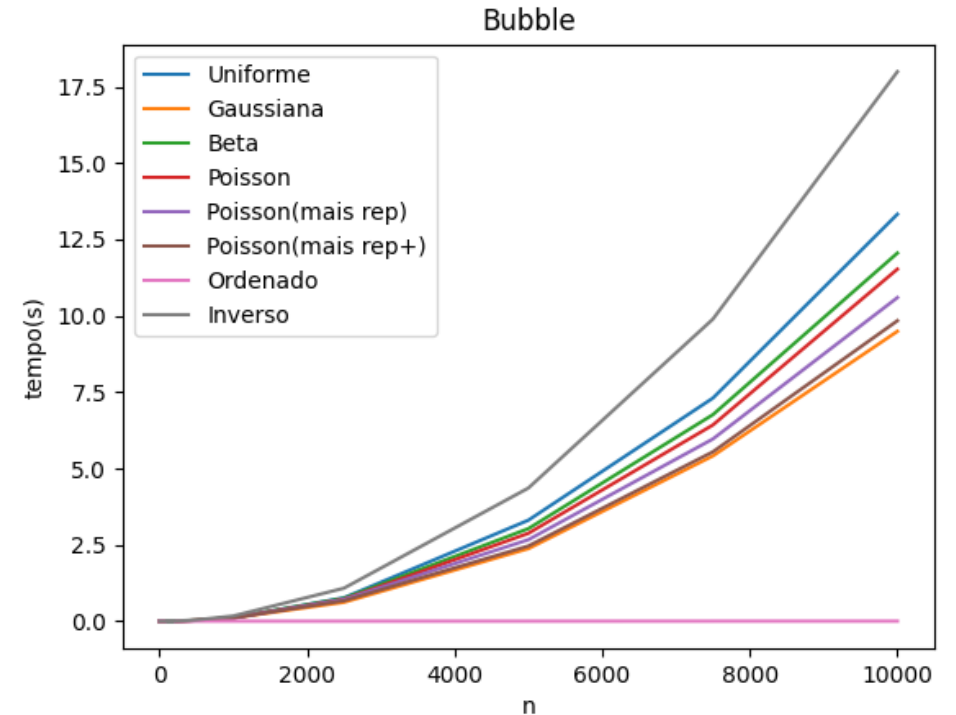
PIOR CASO: INVERSO - $O(N^2)$

CASO MÉDIO: $O(N^2)$

- CÓDIGO:

```
def bubbleSort(V):
    j = 0
    ok = False
    N = len(V)
    while not ok:
        ok = True
        for i in range(N - 1, j, -1):
            if V[i - 1] > V[i]:
                V[i - 1], V[i] = V[i], V[i - 1]
                ok = False
        j += 1
```

GRÁFICO:



QUICK COM PIVÔ FIXO -

- CÓDIGO:

```
def quickSort(V, i, f):  
    if i < f:  
        pivot = partition(V, i, f)  
        quickSort(V, i, pivot - 1)  
        quickSort(V, pivot + 1, f)
```

```
def partition(V, i, f):  
    pivot = V[i]  
    a = i  
    for b in range(i + 1, f + 1):  
        if V[b] <= pivot:  
            a += 1  
            V[a], V[b] = V[b], V[a]  
    V[a], V[i] = V[i], V[a]  
    return a
```

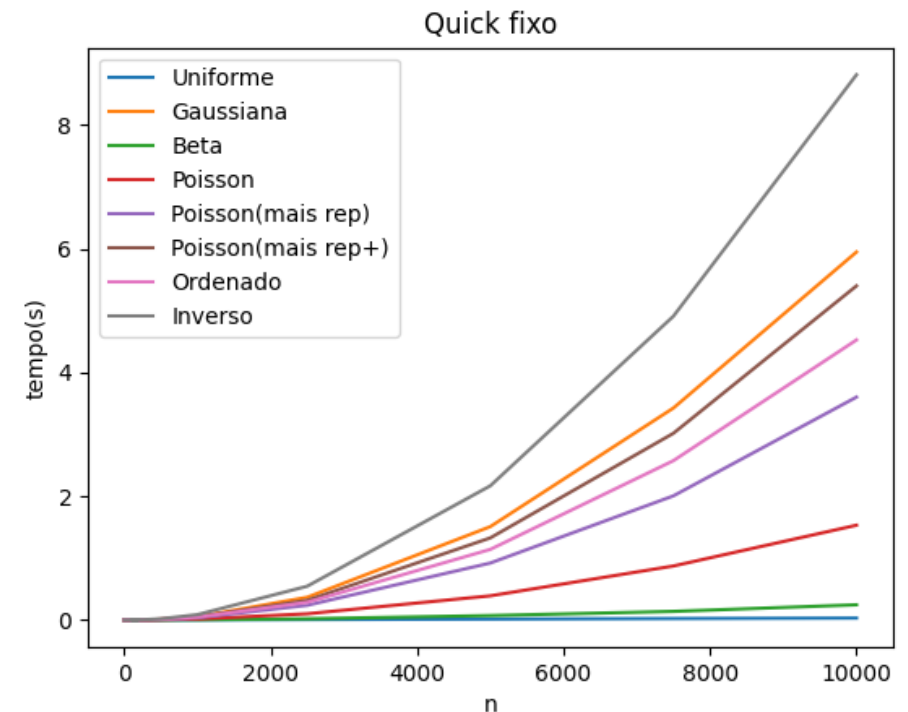
MELHOR CASO:

ORDENADO - $O(N \log N)$

PIOR CASO: INVERSO - $O(N^2)$

CASO MÉDIO: $O(N \log N)$

GRÁFICO:



QUICKSORT COM PIVÔ- ALEATÓRIO

- CÓDIGO:

```
def quickSort2(V, i, f):  
    if i < f:  
        pivot = partition2(V, i, f)  
        quickSort2(V, i, pivot - 1)  
        quickSort2(V, pivot + 1, f)
```

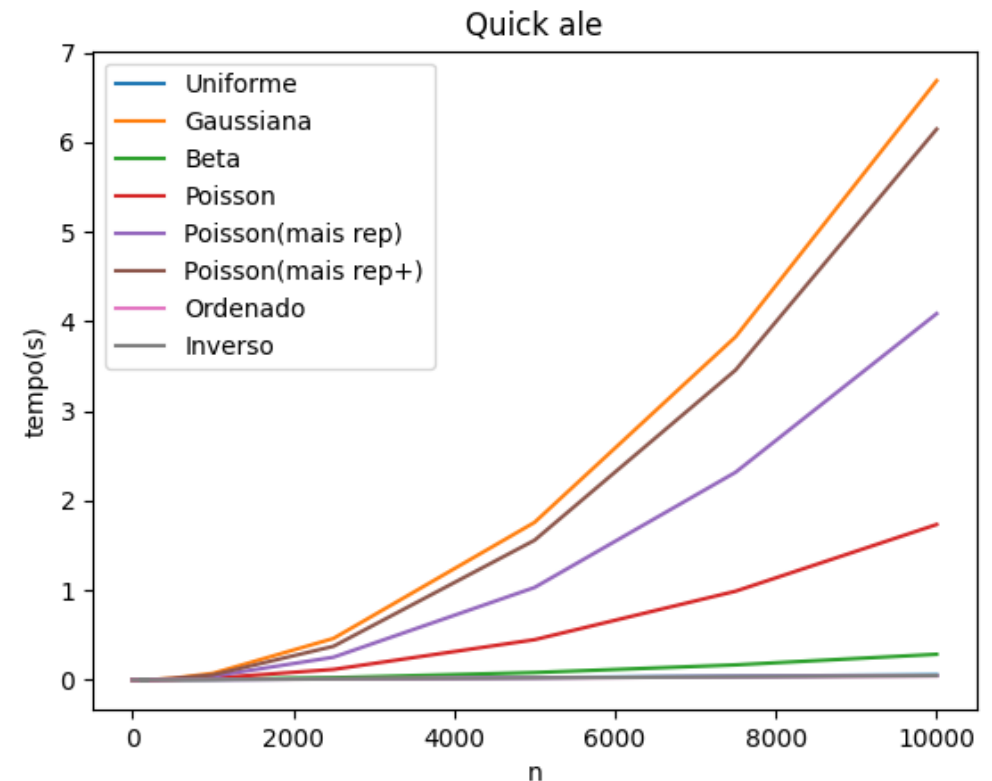
```
def partition2(V, i, f):  
    p = V[i]  
    a = i  
    b = f + 1  
    while True:  
        while True:  
            a += 1  
            if a > f or V[a] >= p:  
                break  
        while True:  
            b -= 1  
            if b < i or V[b] <= p:  
                break  
        if a >= b:  
            break  
        V[a], V[b] = V[b], V[a]  
    V[b], V[i] = V[i], V[b]  
    return b
```

MELHOR CASO: ORDENADO - $O(N\log N)$

PIOR CASO: INVERSO - $O(N^2)$

CASO MÉDIO: $O(N\log N)$

GRÁFICO:



QUICK DE 3 PARTIÇÕES:

```
def quickSort3(A, i, f):  
    if i >= f:  
        return  
    k = random.randint(i, f)  
    A[k], A[i] = A[i], A[k]  
    me, ma = partition3(A, i, f)  
    quickSort3(A, i, me - 1)  
    quickSort3(A, ma + 1, f)
```

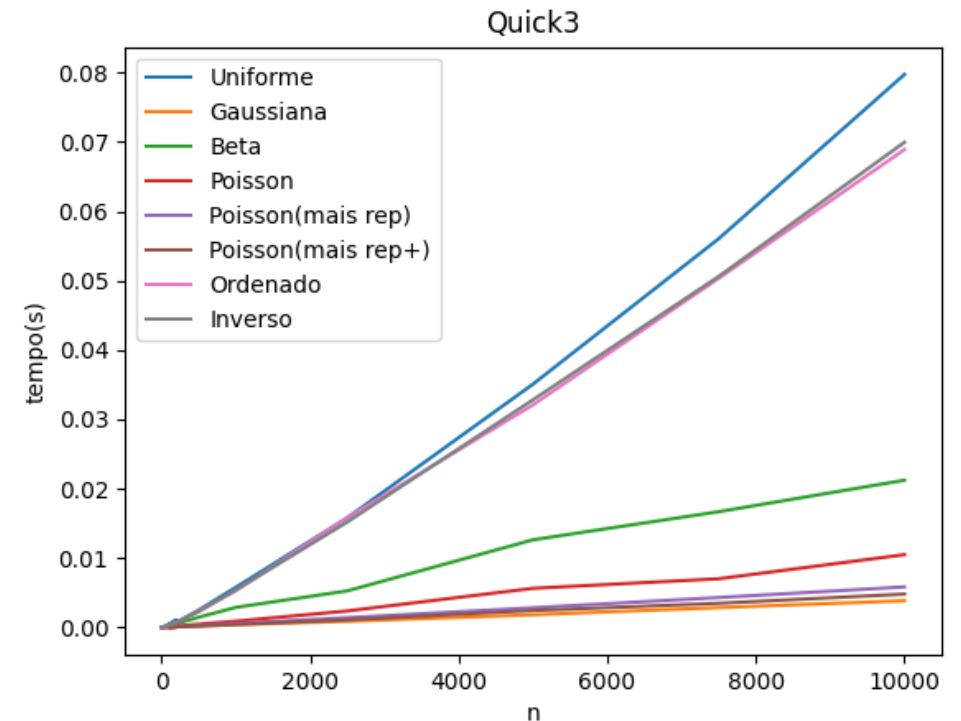
```
def partition3(A, i, f):  
    pivot = A[i]  
    menors = i  
    x = i  
    maiors = f  
  
    while x <= maiors:  
        if A[x] < pivot:  
            A[menors], A[x] = A[x], A[menors]  
            menors += 1  
            x += 1  
        elif A[x] > pivot:  
            A[x], A[maiors] = A[maiors], A[x]  
            maiors -= 1  
        else:  
            x += 1  
    return menors, maiors
```

MELHOR CASO: ORDENADO - $O(N \log N)$

PIOR CASO: INVERSO - $O(N^2)$

CASO MÉDIO: $O(N \log N)$

GRÁFICO:



COUNTING SORT:

- CÓDIGO:

```
def countingSort(A, N, k):
    B = [0]
    for i in range(1,N+1):
        B += [0]
    C = [0]
    for i in range(1,k+1):
        C += [0]
    for i in range(1,N+1):
        C[A[i]] += 1
    for i in range(1,k+1):
        C[i] += C[i-1]
    for j in range(N,0,-1):
        B[C[A[j]]] = A[j]
        C[A[j]] -= 1
    return B
```

MELHOR CASO:

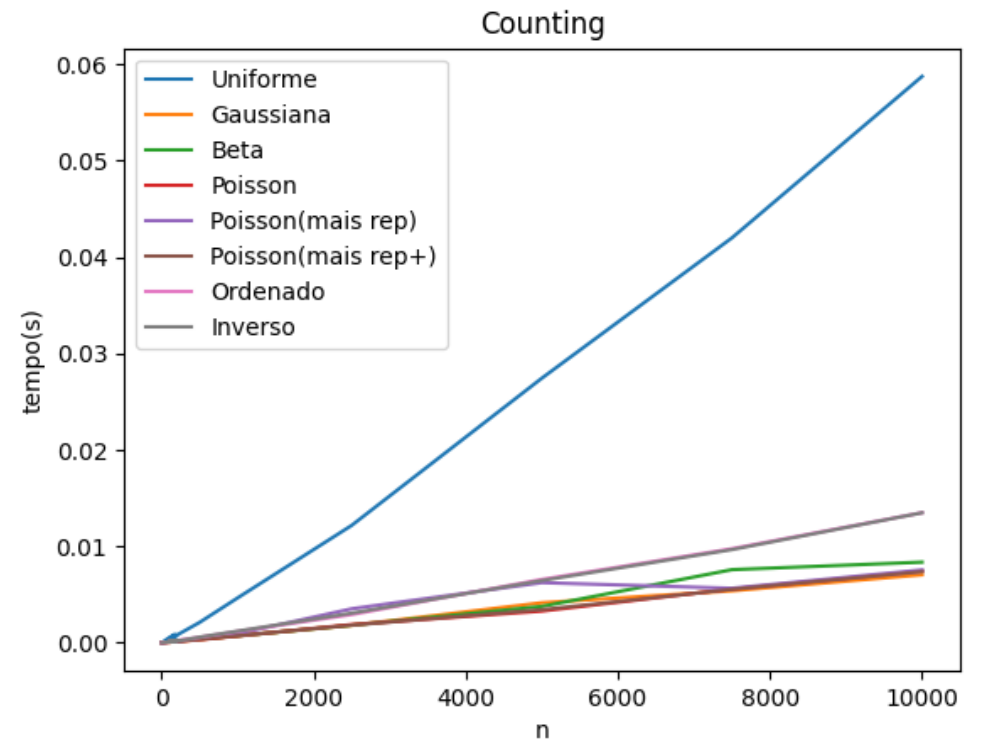
ORDENADO $O(N + M)$

PIOR CASO: INVERSO - $O(N + M)$

CASO MÉDIO: $O(N + M)$

Onde: N é o inputArray[] e M é o countArray[]

GRÁFICO:



CONCLUSÕES